

TRAVAUX PRATIQUES

Système Linux : Administration et Scripts

 **Guide Pratique avec Captures d'Écran** 

Nom & Prénom : Ivan Ehouman Moreno

Date : 04-02-2026

Table des Matières

PARTIE 1 : Le Système Linux

- A. Arborescence et types de fichiers
- B. Lignes de commandes essentielles
- C. Création de fichiers avec touch
- D. Redirections et pipes
- E. Gestion des processus

PARTIE 2 : Scripts Shell

- Exercice 01 : Automatisation avec scripts
- Exercice 02 : Partitionnement de disques

PARTIE 1 : Le Système Linux

A. Arborescence et Types de Fichiers

1. Définition de Linux

Linux est un système d'exploitation open-source de type Unix, créé par Linus Torvalds en 1991. C'est un noyau (kernel) qui sert de pont entre le matériel et les logiciels.

💡 **Astuce** : Linux se distingue par sa philosophie « tout est fichier » et sa structure hiérarchique d'arborescence.

📷 Capture de l'arborescence Linux (commande : tree ou ls /)

```
ivan@Srvubu22-virtual-machine:~$ ls /  
bin      dev      lib      libx32   mnt      root     snap     sys      var  
boot     etc      lib32    lost+found  opt      run      srv      tmp  
cdrom    home     lib64    media    proc     sbin     swapfile usr  
ivan@Srvubu22-virtual-machine:~$
```

2. Pourquoi Linux est un OS orienté fichier

Dans Linux, tout est représenté comme un fichier : les périphériques (disques, clavier), les processus, les sockets réseau. Cette approche uniforme facilite la manipulation et l'administration du système.

- Les périphériques se trouvent dans /dev/
- Les fichiers de configuration dans /etc/
- Les données utilisateur dans /home/

3. Types de fichiers dans Linux

Linux utilise 7 types de fichiers principaux, identifiables par leur premier caractère dans ls -l :

Symbole	Type	Description
---------	------	-------------

-	Fichier régulier	Documents, images, exécutables
d	Répertoire	Dossier contenant d'autres fichiers
l	Lien symbolique	Raccourci vers un autre fichier
b	Fichier bloc	Périphérique bloc (disques durs)
c	Fichier caractère	Périphérique caractère (clavier, souris)
p	Tube nommé (pipe)	Communication entre processus
s	Socket	Communication réseau

📸 Capture montrant les différents types de fichiers (ls -l)

```
ivan@srvubu22-virtual-machine:~$ ls -l
total 36
drwxr-xr-x 2 ivan ivan 4096 févr.  4 16:20 Bureau
drwxr-xr-x 2 ivan ivan 4096 févr.  4 16:20 Documents
drwxr-xr-x 2 ivan ivan 4096 févr.  4 16:20 Images
drwxr-xr-x 2 ivan ivan 4096 févr.  4 16:20 Modèles
drwxr-xr-x 2 ivan ivan 4096 févr.  4 16:20 Musique
drwxr-xr-x 2 ivan ivan 4096 févr.  4 16:20 Public
drwx----- 3 ivan ivan 4096 févr.  4 16:20 snap
drwxr-xr-x 2 ivan ivan 4096 févr.  4 16:20 Téléchargements
drwxr-xr-x 2 ivan ivan 4096 févr.  4 16:20 Vidéos
```

B. Lignes de Commandes Essentielles

Cette section présente les commandes fondamentales pour naviguer et administrer un système Linux. Testez chacune et documentez les résultats avec des captures d'écran.

Commandes d'Information Système

 **Commande :** `uname -a`

 **Description :** Affiche toutes les informations système : nom du noyau, nom de la machine, version du noyau, architecture matérielle, système d'exploitation.

📸 Résultat de `uname -a`

```
root@Srvubu22-virtual-machine: /home/ivan
root@Srvubu22-virtual-machine:/home/ivan# uname -a
Linux Srvubu22-virtual-machine 6.8.0-94-generic #96~22.04.1-Ubuntu SMP PREEMPT_D
YNAMIC Fri Jan 16 13:19:05 UTC 2 x86_64 x86_64 x86_64 GNU/Linux
root@Srvubu22-virtual-machine:/home/ivan#
```

 **Commande :** pwd

 **Description :** Print Working Directory - Affiche le chemin absolu du répertoire courant.

Résultat de pwd

```
root@Srvubu22-virtual-machine:/home/ivan# pwd
/home/ivan
root@Srvubu22-virtual-machine:/home/ivan#
```

 **Commande :** whoami

 **Description :** Affiche le nom de l'utilisateur connecté actuellement.

Résultat de whoami

```
root@Srvubu22-virtual-machine: /home/ivan
root@Srvubu22-virtual-machine:/home/ivan# uname -a
Linux Srvubu22-virtual-machine 6.8.0-94-generic #96~22.04.1-Ubuntu SMP PREEMPT_D
YNAMIC Fri Jan 16 13:19:05 UTC 2 x86_64 x86_64 x86_64 GNU/Linux
root@Srvubu22-virtual-machine:/home/ivan# whoami
root
root@Srvubu22-virtual-machine:/home/ivan#
```

 **Commande :** `who`

 **Description :** Liste tous les utilisateurs actuellement connectés au système avec leur terminal et heure de connexion.

Résultat de who

```
root@Srvubu22-virtual-machine:/home/ivan# who
ivan      tty2          2026-02-05 10:09 (tty2)
ivan      pts/1         2026-02-06 09:30
root@Srvubu22-virtual-machine:/home/ivan#
```

 **Commande :** `which [commande]`

 **Description :** Affiche le chemin complet d'un exécutable. Exemple : `which ls`, `which python`.

 **Astuce :** Utilisez `‘which’` pour vérifier quelle version d'un programme sera exécutée si plusieurs sont installées.

Commandes de Listage (ls et ses options)

 **Commande :** `ls -l`

 **Description :** Affichage détaillé (long format) : permissions, propriétaire, taille, date de modification.

Résultat de ls -l

```
root@Srvubu22-virtual-machine:/home/ivan# ls -l
total 36
drwxr-xr-x 2 ivan ivan 4096 févr.  4 16:20 Bureau
drwxr-xr-x 2 ivan ivan 4096 févr.  4 16:20 Documents
drwxr-xr-x 2 ivan ivan 4096 févr.  4 16:20 Images
drwxr-xr-x 2 ivan ivan 4096 févr.  4 16:20 Modèles
drwxr-xr-x 2 ivan ivan 4096 févr.  4 16:20 Musique
drwxr-xr-x 2 ivan ivan 4096 févr.  4 16:20 Public
drwx----- 3 ivan ivan 4096 févr.  4 16:20 snap
drwxr-xr-x 2 ivan ivan 4096 févr.  4 16:20 Téléchargements
drwxr-xr-x 2 ivan ivan 4096 févr.  4 16:20 Vidéos
root@Srvubu22-virtual-machine:/home/ivan#
```

 **Commande :** `ls -a`

 **Description :** Affiche TOUS les fichiers, y compris les fichiers cachés (commençant par un point).

 **Astuce :** Les fichiers cachés sous Linux commencent par un point (.) comme `.bashrc` ou `.ssh`

Résultat de ls -a montrant les fichiers cachés

```
root@Srvubu22-virtual-machine:/home/ivan# ls -a
.          .bashrc  Documents  Musique   .sudo_as_admin_successful
..         Bureau  Images     .profile  Téléchargements
.bash_history .cache  .local     Public    Vidéos
.bash_logout .config Modèles    snap
root@Srvubu22-virtual-machine:/home/ivan#
```

 **Commande :** `ll`

 **Description :** Alias de `ls -la` (dépend de la distribution). Combine affichage détaillé et fichiers cachés.

 **Commande :** `ls -h`

 **Description :** Human-readable : affiche les tailles en format lisible (Ko, Mo, Go) au lieu d'octets.

Résultat de `ls -lh` montrant les tailles lisibles

```
root@Srvubu22-virtual-machine:/home/ivan# ls -h
Bureau   Images  Musique snap      Vidéos
Documents Modèles Public  Téléchargements
```

 **Commande :** `ls -r`

 **Description :** Reverse : inverse l'ordre de tri (Z à A ou du plus récent au plus ancien).

 **Commande :** `ls -S`

 **Description :** Trie par taille de fichier (du plus grand au plus petit).

 **Commande :** `ls -t`

 **Description :** Trie par date de modification (du plus récent au plus ancien).

 **Commande :** `ls -la`

 **Description :** Combine -l (détaillé) et -a (tout afficher). Format le plus complet pour voir tous les fichiers.

Résultat de `ls -la` avec analyse des permissions


```

root@Srvubu22-virtual-machine:/home/ivan# ls -la
total 72
drwxr-x--- 14 ivan ivan 4096 févr.  5 10:08 .
drwxr-xr-x  3 root root 4096 févr.  4 16:12 ..
-rw-----  1 ivan ivan   8 févr.  5 10:08 .bash_history
-rw-r--r--  1 ivan ivan 220 févr.  4 16:12 .bash_logout
-rw-r--r--  1 ivan ivan 3771 févr.  4 16:12 .bashrc
drwxr-xr-x  2 ivan ivan 4096 févr.  4 16:20 Bureau
drwx----- 10 ivan ivan 4096 févr.  4 16:24 .cache
drwx----- 12 ivan ivan 4096 févr.  5 10:08 .config
drwxr-xr-x  2 ivan ivan 4096 févr.  4 16:20 Documents
drwxr-xr-x  2 ivan ivan 4096 févr.  4 16:20 Images
drwx-----  3 ivan ivan 4096 févr.  4 16:20 .local
drwxr-xr-x  2 ivan ivan 4096 févr.  4 16:20 Modèles
drwxr-xr-x  2 ivan ivan 4096 févr.  4 16:20 Musique
-rw-r--r--  1 ivan ivan  807 févr.  4 16:12 .profile
drwxr-xr-x  2 ivan ivan 4096 févr.  4 16:20 Public
drwx-----  3 ivan ivan 4096 févr.  4 16:20 snap
-rw-r--r--  1 ivan ivan   0 févr.  4 16:21 .sudo_as_admin_successful
drwxr-xr-x  2 ivan ivan 4096 févr.  4 16:20 Téléchargements
drwxr-xr-x  2 ivan ivan 4096 févr.  4 16:20 Vidéos
root@Srvubu22-virtual-machine:/home/ivan#

```

 **Commande :** `ls -lhS`

 **Description :** Affichage détaillé, tailles lisibles, trié par taille. Pratique pour trouver les gros fichiers.

 **Astuce :** Combinez les options selon vos besoins : `ls -lhSr` affichera du plus petit au plus grand fichier.

Commandes de Navigation (cd)

 **Commande :** `cd /`

 **Description :** Change directory vers la racine du système de fichiers (le répertoire racine).

Navigation vers / et affichage de son contenu

```

root@Srvubu22-virtual-machine:/home/ivan# cd /
root@Srvubu22-virtual-machine:/#

```

 **Commande :** `cd ~`

 **Description :** Retourne au répertoire personnel (home) de l'utilisateur. Équivaut à `cd $HOME` ou `cd` sans argument.

Navigation vers ~ (home)

```
root@Srvubu22-virtual-machine:/# cd ~  
root@Srvubu22-virtual-machine:~#
```

 **Commande :** `cd ..`

 **Description :** Remonte d'un niveau dans l'arborescence (répertoire parent).

 **Astuce :** Utilisez `cd ../../` pour remonter de deux niveaux d'un coup.

 **Commande :** `cd .`

 **Description :** Reste dans le répertoire courant (`.` représente le répertoire actuel). Peu utilisé seul.

Commandes de Gestion de Disques et Espace

 **Commande :** `uptime`

 **Description :** Affiche depuis combien de temps le système fonctionne, nombre d'utilisateurs et charge moyenne.

Résultat de uptime

```
root@Srvubu22-virtual-machine:~# uptime  
15:45:47 up 10:28, 2 users, load average: 0,00, 0,04, 0,01  
root@Srvubu22-virtual-machine:~#
```

 **Commande :** `du`

 **Description :** Disk Usage - Estime l'espace disque utilisé par les fichiers et répertoires.

 **Commande :** `du -hc --max-depth=1`

 **Description :** Affiche l'espace utilisé par chaque sous-répertoire direct (-h lisible, -c total, --max-depth=1 un niveau).

Résultat de `du -hc --max-depth=1` dans `/home`

```
root@Srvubu22-virtual-machine:~# uptime
15:45:47 up 10:28,  2 users,  load average: 0,00, 0,04, 0,01
root@Srvubu22-virtual-machine:~# du -hc --max-depth=1
48K      ./snap
4,0K     ./cache
64K      .
64K      total
root@Srvubu22-virtual-machine:~#
```

 **Commande :** `du -sh dossier1 dossier2`

 **Description :** Affiche la taille totale de chaque dossier spécifié (-s summary, -h human-readable).

 **Astuce :** Très utile pour identifier rapidement les dossiers qui occupent le plus d'espace.

 **Commande :** `df`

 **Description :** Disk Free - Affiche l'espace disponible sur tous les systèmes de fichiers montés.

Résultat de `df -h`

```
root@Srvubu22-virtual-machine:~# df
Sys. de fichiers blocs de 1K Utilisé Disponible Uti% Monté sur
tmpfs                396132      1952    394180    1% /run
/dev/sda3            19946096 12878924    6028632   69% /
tmpfs                1980640         0    1980640    0% /dev/shm
tmpfs                 5120         4      5116    1% /run/lock
/dev/sda2             524252     6232    518020    2% /boot/efi
tmpfs                396128     112    396016    1% /run/user/1000
root@Srvubu22-virtual-machine:~#
```

 **Commande :** `fdisk -l`

 **Description :** Liste tous les disques et leurs partitions (nécessite les droits root avec sudo).

Résultat de sudo fdisk -l

```
Disque /dev/loop11 : 452 KiB, 462848 octets, 904 secteurs
Unités : secteur de 1 × 512 = 512 octets
Taille de secteur (logique / physique) : 512 octets / 512 octets
taille d'E/S (minimale / optimale) : 512 octets / 512 octets

Disque /dev/loop12 : 576 KiB, 589824 octets, 1152 secteurs
Unités : secteur de 1 × 512 = 512 octets
Taille de secteur (logique / physique) : 512 octets / 512 octets
taille d'E/S (minimale / optimale) : 512 octets / 512 octets

Disque /dev/loop13 : 73,96 MiB, 77549568 octets, 151464 secteurs
Unités : secteur de 1 × 512 = 512 octets
Taille de secteur (logique / physique) : 512 octets / 512 octets
taille d'E/S (minimale / optimale) : 512 octets / 512 octets

Disque /dev/loop14 : 63,77 MiB, 66871296 octets, 130608 secteurs
Unités : secteur de 1 × 512 = 512 octets
Taille de secteur (logique / physique) : 512 octets / 512 octets
taille d'E/S (minimale / optimale) : 512 octets / 512 octets
root@Srvubu22-virtual-machine:~#
```

 **Commande :** cfdisk

 **Description :** Éditeur de partitions en mode texte interactif (plus convivial que fdisk).

 **Astuce :** ⚠ ATTENTION : cfdisk modifie les partitions. Ne l'utilisez qu'avec précaution !

Commandes de Visualisation de Fichiers

 **Commande :** cat fichier.txt

 **Description :** Affiche tout le contenu d'un fichier d'un coup. Idéal pour les petits fichiers.

Utilisation de cat sur un fichier

```
root@Srvubu22-virtual-machine:~# ^[[200~cat fichier.txt
cat : commande introuvable
root@Srvubu22-virtual-machine:~# ~
```

 **Commande :** less fichier.txt

 **Description :** Affiche le contenu page par page (navigation avec flèches, q pour quitter).

 **Astuce :** Dans less : espace = page suivante, b = page précédente, / = rechercher, q = quitter.

Utilisation de less

Commandes de Recherche

 **Commande :** find /chemin -name "*.txt"

 **Description :** Recherche des fichiers par nom. Exemple : find /home -name "rapport*" pour tous les fichiers commençant par ‘rapport’.

Utilisation de find

```
root@srvubu22-virtual-machine:~# find .
./snap
./snap/snap-store
./snap/snap-store/959
./snap/snap-store/current
./snap/snap-store/common
./snap/firefox
./snap/firefox/2987
./snap/firefox/current
./snap/firefox/common
./snap/firefox/7766
./snap/snapd-desktop-integration
./snap/snapd-desktop-integration/83
./snap/snapd-desktop-integration/current
./snap/snapd-desktop-integration/common
./snap/snapd-desktop-integration/343
./.cache
./.profile
./.bashrc
root@srvubu22-virtual-machine:~#
```

 **Commande :** `grep "motif" fichier.txt`

 **Description :** Recherche un motif (pattern) dans un fichier. Exemple : `grep "error" logfile.log`.

 **Astuce :** Combinez find et grep : `find . -name "*.log" -exec grep "error" {} \;`

Utilisation de grep pour chercher dans des fichiers

[Insérez votre capture d'écran ici]

Commande de Montage

 **Commande :** `mount`

 **Description :** Sans arguments : affiche tous les systèmes de fichiers montés.
Avec arguments : monte un système de fichiers.

Résultat de la commande mount

```
root@Srvubu22-virtual-machine: ~
ev,relatime,errors=continue,threads=single,x-gdu.hide)
/dev/sda3 on /var/snap/firefox/common/host-hunspell type ext4 (ro,noexec,noatime
,errors=remount-ro)
/var/lib/snapd/snaps/snapd-desktop-integration_343.snap on /snap/snapd-desktop-i
ntegration/343 type squashfs (ro,nodev,relatime,errors=continue,threads=single,x
-gdu.hide)
/dev/sda2 on /boot/efi type vfat (rw,relatime,fmask=0077,dmask=0077,codepage=437
,iocharset=iso8859-1,shortname=mixed,errors=remount-ro)
binfmt_misc on /proc/sys/fs/binfmt_misc type binfmt_misc (rw,nosuid,nodev,noexec
,relatime)
tmpfs on /run/snapd/ns type tmpfs (rw,nosuid,nodev,noexec,relatime,size=396132k,
mode=755,inode64)
nsfs on /run/snapd/ns/snapd-desktop-integration.mnt type nsfs (rw)
tmpfs on /run/user/1000 type tmpfs (rw,nosuid,nodev,relatime,size=396128k,nr_ino
des=99032,mode=700,uid=1000,gid=1000,inode64)
portal on /run/user/1000/doc type fuse.portal (rw,nosuid,nodev,relatime,user_id=
1000,group_id=1000)
gvfsd-fuse on /run/user/1000/gvfs type fuse.gvfsd-fuse (rw,nosuid,nodev,relatime
,user_id=1000,group_id=1000)
/var/lib/snapd/snaps/core22_2292.snap on /snap/core22/2292 type squashfs (ro,nod
ev,relatime,errors=continue,threads=single,x-gdu.hide)
/var/lib/snapd/snaps/core20_2686.snap on /snap/core20/2686 type squashfs (ro,nod
ev,relatime,errors=continue,threads=single,x-gdu.hide)
root@Srvubu22-virtual-machine:~#
```

C. Création de Fichiers avec touch

1. Créer un fichier invisible

 **Commande :** touch .monfichier

 **Description :** Les fichiers cachés commencent par un point (.). Pour vérifier : ls -a

Création d'un fichier caché et vérification avec ls -a

```
root@Srvubu22-virtual-machine:~# ls -a
.  ..  .bashrc  .cache  .monfichier  .profile  snap
root@Srvubu22-virtual-machine:~#
```

2. Créer un fichier vide

 **Commande :** touch fichier_vide.txt

 **Description :** Crée un fichier sans contenu ou met à jour la date de modification

s'il existe déjà.

💡 **Astuce** : touch peut créer plusieurs fichiers d'un coup : touch fichier1.txt fichier2.txt fichier3.txt

📹 Création de fichiers avec touch et vérification

```
root@Srvubu22-virtual-machine:~# touch fichier_vide.txt
root@Srvubu22-virtual-machine:~#
```

D. Redirections et Manipulation de Fichiers avec echo

1. Créer des fichiers avec du contenu

💻 **Commande** : echo "merci" > test1.txt

📁 **Description** : Crée test1.txt contenant ‘merci’ (> écrase le fichier s'il existe).

💻 **Commande** : echo "Aurevoir" > test2.txt

📁 **Description** : Crée test2.txt contenant ‘Aurevoir’.

💡 **Astuce** : Différence : > écrase le fichier, >> ajoute à la fin (append).

📹 Création de test1.txt et test2.txt, puis affichage avec cat

```
root@Srvubu22-virtual-machine:~# echo "merci" > test1.txt
root@Srvubu22-virtual-machine:~# echo "Aurevoir" > test2.txt
root@Srvubu22-virtual-machine:~#
```

2. Combiner le contenu de fichiers

Pour créer un fichier qui commence par ‘Bonjour’ et contient ensuite le contenu de test1.txt et test2.txt :

 **Commande :** `echo "Bonjour" > fichier_combine.txt`

`cat test1.txt >> fichier_combine.txt`

`cat test2.txt >> fichier_combine.txt`

 **Description :** Première ligne : crée le fichier avec `‘Bonjour’`. Lignes suivantes : ajoutent (`>>`) le contenu des autres fichiers.

Création du fichier combiné et affichage de son contenu

```
root@Srvubu22-virtual-machine:~# echo "Bonjour" > fichier_combine.txt
cat test1.txt >> fichier_combine.txt
cat test2.txt >> fichier_combine.txt
root@Srvubu22-virtual-machine:~#
```

3. Copier et ajouter du contenu

 **Commande :** `cat fichier_combine.txt > copie_fichier.txt`

`echo "stop" >> copie_fichier.txt`

 **Description :** Première ligne : copie le contenu. Deuxième ligne : ajoute `‘stop’` à la fin.

 **Astuce :** Alternative avec `cp` : `cp fichier_combine.txt copie_fichier.txt` puis `echo "stop" >> copie_fichier.txt`

Création de la copie et ajout du mot 'stop'

```
root@Srvubu22-virtual-machine:~# : cat fichier_combine.txt > copie_fichier.txt
echo "stop" >> copie_fichier.txt
root@Srvubu22-virtual-machine:~#
```

E. Gestion des Processus

1. Commande ps (Process Status)

 **Commande :** `ps`

 **Description :** Affiche les processus de l'utilisateur courant dans le terminal actuel.

 **Commande :** `ps aux`

 **Description :** Affiche TOUS les processus de tous les utilisateurs avec détails (a=all, u=user-oriented, x=include processes without TTY).

 **Commande :** `ps -ef`

 **Description :** Format standard UNIX pour voir tous les processus.

Résultat de `ps aux` montrant les processus actifs

```
root      5104      1  0  09:50 ?        00:00:00 sshd: /usr/sbin/sshd -D [lis
root      5294      2  0  10:00 ?        00:00:00 [kworker/1:1H-kblockd]
root      5485      2  0  10:14 ?        00:00:00 [kworker/u261:0-ttm]
root      5487      2  0  10:14 ?        00:00:00 [kworker/u260:2-ttm]
root      5609      2  0  10:23 ?        00:00:10 [kworker/1:2-events]
root      6173      2  0  11:55 ?        00:00:00 [kworker/0:0H]
root      6235      2  0  13:40 ?        00:00:00 [kworker/u258:4-events_unbou
root      6243      2  0  14:03 ?        00:00:00 [kworker/u257:0-events_unbou
root      6281      2  0  15:12 ?        00:00:03 [kworker/0:0-events]
root      6283      2  0  15:12 ?        00:00:00 [kworker/u260:1-ttm]
root      6347      1  0  15:13 ?        00:00:01 /usr/libexec/fwupd/fwupd
root      6394      1  0  15:13 ?        00:00:00 /usr/sbin/cupsd -l
root      6401      1  0  15:13 ?        00:00:00 /usr/sbin/cups-browsed
root      8274      2  0  15:13 ?        00:00:00 [kworker/0:2H-kblockd]
root      9188      2  0  15:22 ?        00:00:00 [kworker/1:1-cgroup_destroy]
root      9648      2  0  15:32 ?        00:00:00 [kworker/0:1-rcu_par_gp]
ivan      9677    1632  0  15:32 ?        00:00:01 gjs /usr/share/gnome-shell/e
root      9737      2  0  15:44 ?        00:00:00 [kworker/u257:2-flush-8:0]
root      9738      2  0  15:44 ?        00:00:00 [kworker/u258:0-events_unbou
root      9741      2  0  15:45 ?        00:00:00 [kworker/u258:1-events_unbou
root      9773      2  0  15:51 ?        00:00:00 [kworker/u257:3-events_power
root      9799      2  0  15:58 ?        00:00:00 [kworker/u257:1-events_unbou
root      9801    4312  0  15:58 pts/1    00:00:00 ps -ef
root@Srvubu22-virtual-machine:~#
```

2. Commande `top` (Monitoring en temps réel)

 **Commande :** `top`

 **Description :** Affichage dynamique des processus avec mise à jour en temps réel (q pour quitter).

 **Astuce :** Dans `top` : M = trier par mémoire, P = trier par CPU, k = tuer un

processus, q = quitter.

Interface de top montrant l'utilisation CPU et mémoire

```
top - 15:59:38 up 10:42, 2 users, load average: 0,03, 0,06, 0,02
Tâches: 297 total, 1 en cours, 296 en veille, 0 arrêté, 0 zombie
%Cpu(s): 0,8 ut, 1,2 sy, 0,0 ni, 98,0 id, 0,0 wa, 0,0 hi, 0,0 si, 0,0 st
MiB Mem : 3868,4 total, 1146,2 libr, 979,6 util, 1742,6 tamp/cache
MiB Éch: 2140,0 total, 2140,0 libr, 0,0 util. 2602,2 dispo Mem
```

PID	UTIL.	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TEMPS+	COM.
1632	ivan	20	0	4422416	269060	129308	S	3,3	6,8	2:36.47	gnome-s+
4284	ivan	20	0	558076	53480	40876	S	1,0	1,4	0:08.20	gnome-t+
644	root	20	0	246576	9472	7808	S	0,7	0,2	1:24.67	vmtoolsd
9803	root	20	0	15992	4352	3456	R	0,7	0,1	0:00.04	top
1788	ivan	20	0	149120	44120	30276	S	0,3	1,1	1:17.46	vmtoolsd
1	root	20	0	168052	13116	8252	S	0,0	0,3	0:05.86	systemd
2	root	20	0	0	0	0	S	0,0	0,0	0:00.10	kthreadd
3	root	20	0	0	0	0	S	0,0	0,0	0:00.00	pool_wor
4	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker+
5	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker+
6	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker+
7	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker+
11	root	20	0	0	0	0	I	0,0	0,0	0:00.00	kworker+
12	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker+
13	root	20	0	0	0	0	I	0,0	0,0	0:00.00	rcu_tas+
14	root	20	0	0	0	0	I	0,0	0,0	0:00.00	rcu_tas+
15	root	20	0	0	0	0	I	0,0	0,0	0:00.00	rcu_tas+

3. Tuer un processus

 **Commande :** kill PID

 **Description :** Envoie un signal de terminaison au processus (PID = Process ID trouvé avec ps ou top).

 **Commande :** kill -9 PID

 **Description :** Force la terminaison immédiate du processus (SIGKILL).

 **Commande :** killall nom_processus

 **Description :** Tue tous les processus ayant ce nom.

 **Astuce :** ⚠ Attention : kill -9 ne laisse pas le processus se terminer proprement. À utiliser en dernier recours.

Exemple de kill d'un processus

```
root@Srvubu22-virtual-machine:~# kill PID
bash: kill: PID : les arguments doivent être des identifiants de tâche ou de processus
root@Srvubu22-virtual-machine:~# kill -9 PID
bash: kill: PID : les arguments doivent être des identifiants de tâche ou de processus
root@Srvubu22-virtual-machine:~# killall nom_processus
```

PARTIE 2 : Scripts Shell

Cette partie vous demande de créer des scripts shell pour automatiser des tâches d'administration système. Chaque script doit être documenté avec des commentaires et testé avec des captures d'écran.

💡 **Astuce :** Pensez à rendre vos scripts exécutables avec : `chmod +x nom_script.sh`

Exercice 01 : Scripts d'Automatisation

1. Script de création de répertoires et fichiers

Objectif : Créer dynamiquement minimum 3 répertoires avec leurs sous-répertoires et fichiers.

Exemple de structure :

- REP1/SREP1/fichier1.txt
- REP2/SREP2/fichier2.txt
- REP3/SREP3/fichier3.txt

💡 **Exemple de script (à adapter) :**

```
#!/bin/bash
# Script de création dynamique de répertoires

for i in {1..3}; do
    mkdir -p REP$i/SREP$i
```

```
touch REP$i/SREP$i/fichier$i.txt
echo "Créé : REP$i/SREP$i/fichier$i.txt"
done

echo "Création terminée !"
ls -R REP*
```

Code de votre script dans un éditeur (nano ou vi)

```
root@Srvubu22-virtual-machine:~# #!/bin/bash
# Script de création dynamique de répertoires

for i in {1..3}; do
    mkdir -p REP$i/SREP$i
    touch REP$i/SREP$i/fichier$i.txt
    echo "Créé : REP$i/SREP$i/fichier$i.txt"
done
echo "Création terminée !"
ls -R REP
Créé : REP1/SREP1/fichier1.txt
Créé : REP2/SREP2/fichier2.txt
Créé : REP3/SREP3/fichier3.txt
Création terminée !
```

Exécution du script et résultat (ls -R)

```
2987 7766 common current
./snap/firefox/2987:
./snap/firefox/7766:
./snap/firefox/common:
./snap/snapd-desktop-integration:
343 83 common current
./snap/snapd-desktop-integration/343:
./snap/snapd-desktop-integration/83:
./snap/snapd-desktop-integration/common:
./snap/snap-store:
959 common current
./snap/snap-store/959:
./snap/snap-store/common:
root@Srvubu22-virtual-machine:~#
```

2. Script de création d'utilisateurs rattachés aux dossiers

Objectif : Créer minimum 3 utilisateurs et les rattacher aux répertoires créés précédemment.

Exemple : util1 → REP1, util2 → REP2, util3 → REP3

💡 Exemple de script :

```
#!/bin/bash
# Script de création d'utilisateurs

for i in {1..3}; do
    # Créer l'utilisateur
    sudo useradd -m -s /bin/bash util$i

    # Définir un mot de passe (à adapter)
```



```
echo "util$i:motdepasse" | sudo chpasswd

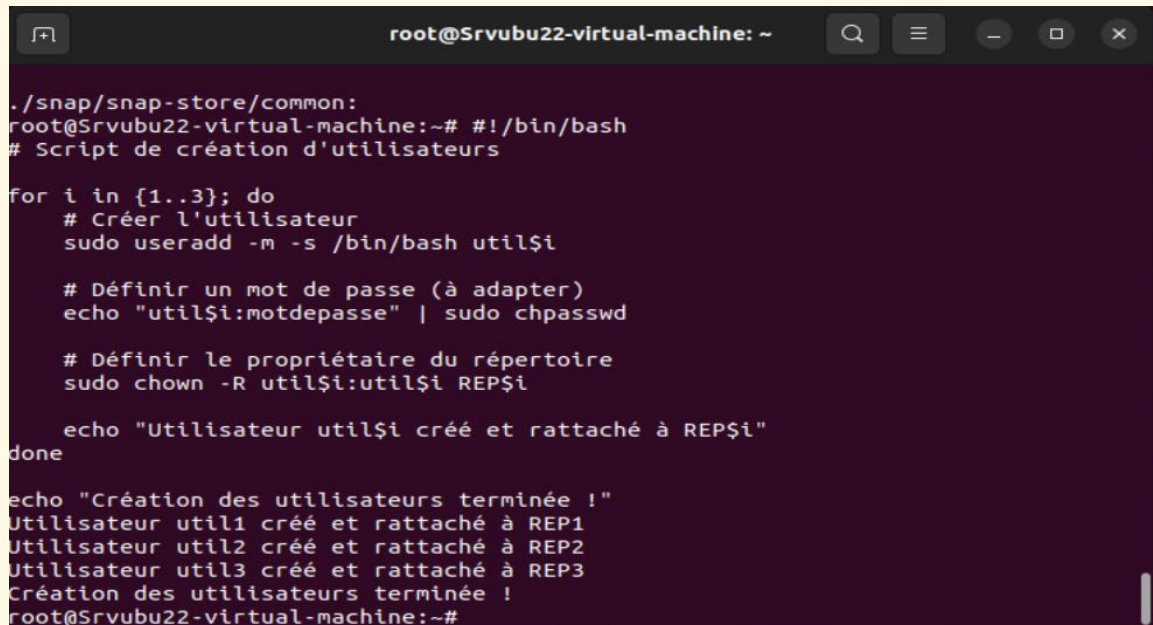
# Définir le propriétaire du répertoire
sudo chown -R util$i:util$i REP$i

echo "Utilisateur util$i créé et rattaché à REP$i"
done

echo "Création des utilisateurs terminée !"
```

💡 **Astuce :** ⚠ Ce script nécessite les droits sudo. Vérifiez les utilisateurs créés avec : `cat /etc/passwd | grep util`

📹 Exécution du script de création d'utilisateurs



The screenshot shows a terminal window titled "root@Srvubu22-virtual-machine: ~". The script being executed is located at `./snap/snap-store/common:`. The script's content is as follows:

```
root@Srvubu22-virtual-machine:~# #!/bin/bash
# Script de création d'utilisateurs

for i in {1..3}; do
    # Créer l'utilisateur
    sudo useradd -m -s /bin/bash util$i

    # Définir un mot de passe (à adapter)
    echo "util$i:motdepasse" | sudo chpasswd

    # Définir le propriétaire du répertoire
    sudo chown -R util$i:util$i REP$i

    echo "Utilisateur util$i créé et rattaché à REP$i"
done

echo "Création des utilisateurs terminée !"
Utilisateur util1 créé et rattaché à REP1
Utilisateur util2 créé et rattaché à REP2
Utilisateur util3 créé et rattaché à REP3
Création des utilisateurs terminée !
root@Srvubu22-virtual-machine:~#
```

📹 Vérification avec `cat /etc/passwd` et `ls -l REP*`


```

root@Srvubu22-virtual-machine: ~
cups-pk-helper:x:115:122:user for cups-pk-helper service,,,:/home/cups-pk-helper
:/usr/sbin/nologin
rtkit:x:116:123:RealtimeKit,,,:/proc:/usr/sbin/nologin
whoopsie:x:117:124::/nonexistent:/bin/false
sssd:x:118:125:SSSD system user,,,:/var/lib/sss:/usr/sbin/nologin
speech-dispatcher:x:119:29:Speech Dispatcher,,,:/run/speech-dispatcher:/bin/false
fwupd-refresh:x:120:126:fwupd-refresh user,,,:/run/systemd:/usr/sbin/nologin
nm-openvpn:x:121:127:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin
/nologin
saned:x:122:129::/var/lib/saned:/usr/sbin/nologin
colord:x:123:130:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/no
login
geoclue:x:124:131::/var/lib/geoclue:/usr/sbin/nologin
pulse:x:125:132:PulseAudio daemon,,,:/run/pulse:/usr/sbin/nologin
gnome-initial-setup:x:126:65534::/run/gnome-initial-setup:/bin/false
hplip:x:127:7:HPLIP system user,,,:/run/hplip:/bin/false
gdm:x:128:134:Gnome Display Manager:/var/lib/gdm3:/bin/false
ivan:x:1000:1000:Ivan,,,:/home/ivan:/bin/bash
sshd:x:129:65534::/run/sshd:/usr/sbin/nologin
util1:x:1001:1001::/home/util1:/bin/bash
util2:x:1002:1002::/home/util2:/bin/bash
util3:x:1003:1003::/home/util3:/bin/bash
root@Srvubu22-virtual-machine:~#

```

3. Script de création de groupes affiliés aux utilisateurs

Objectif : Créer 3 groupes et y affecter les utilisateurs créés précédemment.

💡 Exemple de script :

```

#!/bin/bash
# Script de création de groupes

for i in {1..3}; do
    # Créer le groupe
    sudo groupadd groupe$i

    # Ajouter l'utilisateur au groupe
    sudo usermod -aG groupe$i util$i

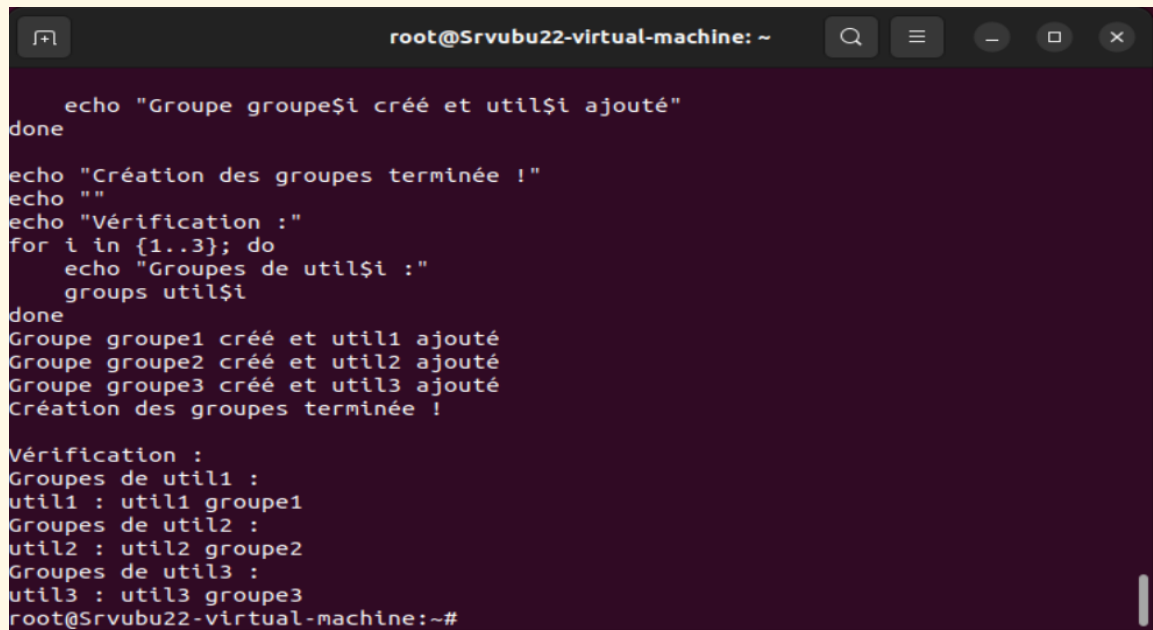
    echo "Groupe groupe$i créé et util$i ajouté"
done

echo "Création des groupes terminée !"
echo ""
echo "Vérification :"
for i in {1..3}; do

```

```
echo "Groupes de util$i :"  
groups util$i  
done
```

📺 Exécution du script et vérification des groupes



```
root@Srvubu22-virtual-machine: ~  
  
    echo "Groupe groupe$i créé et util$i ajouté"  
done  
  
echo "Création des groupes terminée !"  
echo ""  
echo "Vérification :"  
for i in {1..3}; do  
    echo "Groupes de util$i :"  
    groups util$i  
done  
Groupe groupe1 créé et util1 ajouté  
Groupe groupe2 créé et util2 ajouté  
Groupe groupe3 créé et util3 ajouté  
Création des groupes terminée !  
  
Vérification :  
Groupes de util1 :  
util1 : util1 groupe1  
Groupes de util2 :  
util2 : util2 groupe2  
Groupes de util3 :  
util3 : util3 groupe3  
root@Srvubu22-virtual-machine:~#
```

4. Changement de propriétaire d'un dossier

Objectif : Utiliser la commande `chown` pour changer le propriétaire d'un des dossiers créés.

🖥 **Commande :** `sudo chown -R nouvel_utilisateur:nouveau_groupe REP1`

🔧 **Description :** Change le propriétaire et le groupe du dossier REP1 et de tout son contenu (-R = récursif).

💡 **Astuce :** Vérifiez le changement avec : `ls -ld REP1` et `ls -lR REP1`

📺 Commande `chown` et vérification du changement de propriétaire

5. Script de suppression sécurisée

Objectif : Créer un script qui supprime un dossier avec confirmation pour éviter les erreurs.

💡 Exemple de script :

```
#!/bin/bash
# Script de suppression sécurisée

read -p "Entrez le nom du dossier à supprimer : " dossier

if [ ! -d "$dossier" ]; then
    echo "❌ Erreur : Le dossier '$dossier' n'existe pas."
    exit 1
fi

echo ""
echo "📁 Contenu du dossier :"
ls -lR "$dossier"
echo ""

read -p "⚠️ Êtes-vous sûr de vouloir supprimer '$dossier' ? (oui/non) : " reponse

if [ "$reponse" = "oui" ]; then
    rm -rf "$dossier"
    echo "✅ Dossier '$dossier' supprimé avec succès."
else
    echo "❌ Suppression annulée."
fi
```

💡 **Astuce :** Ce script affiche le contenu avant suppression et demande une confirmation explicite.

📺 **Exécution du script de suppression avec confirmation**

```

fi
echo ""
echo "📁 Contenu du dossier :"
ls -lR "$dossier"
echo ""

read -p "⚠️ Êtes-vous sûr de vouloir supprimer '$dossier' ? (oui/non) : " reponse

if [ "$reponse" = "oui" ]; then
    rm -rf "$dossier"
    echo "✅ Dossier '$dossier' supprimé avec succès."
else
    echo "❌ Suppression annulée."
fi

```

Exercice 02 : Partitionnement de Disques

Cet exercice consiste à créer et partitionner un disque virtuel de 15 Go avec 3 partitions différentes.

Objectif complet

- Créer un disque dur virtuel de 15 Go dans VMware
- Partition 1 : 5 Go, type Windows (NTFS), point de montage /mnt/windows
- Partition 2 : 5 Go, type Linux (ext4), point de montage /mnt/linux1
- Partition 3 : 5 Go, type Linux (ext4), point de montage /mnt/linux2
- Formater chaque partition avec son système de fichiers
- Monter les partitions et tester l'écriture

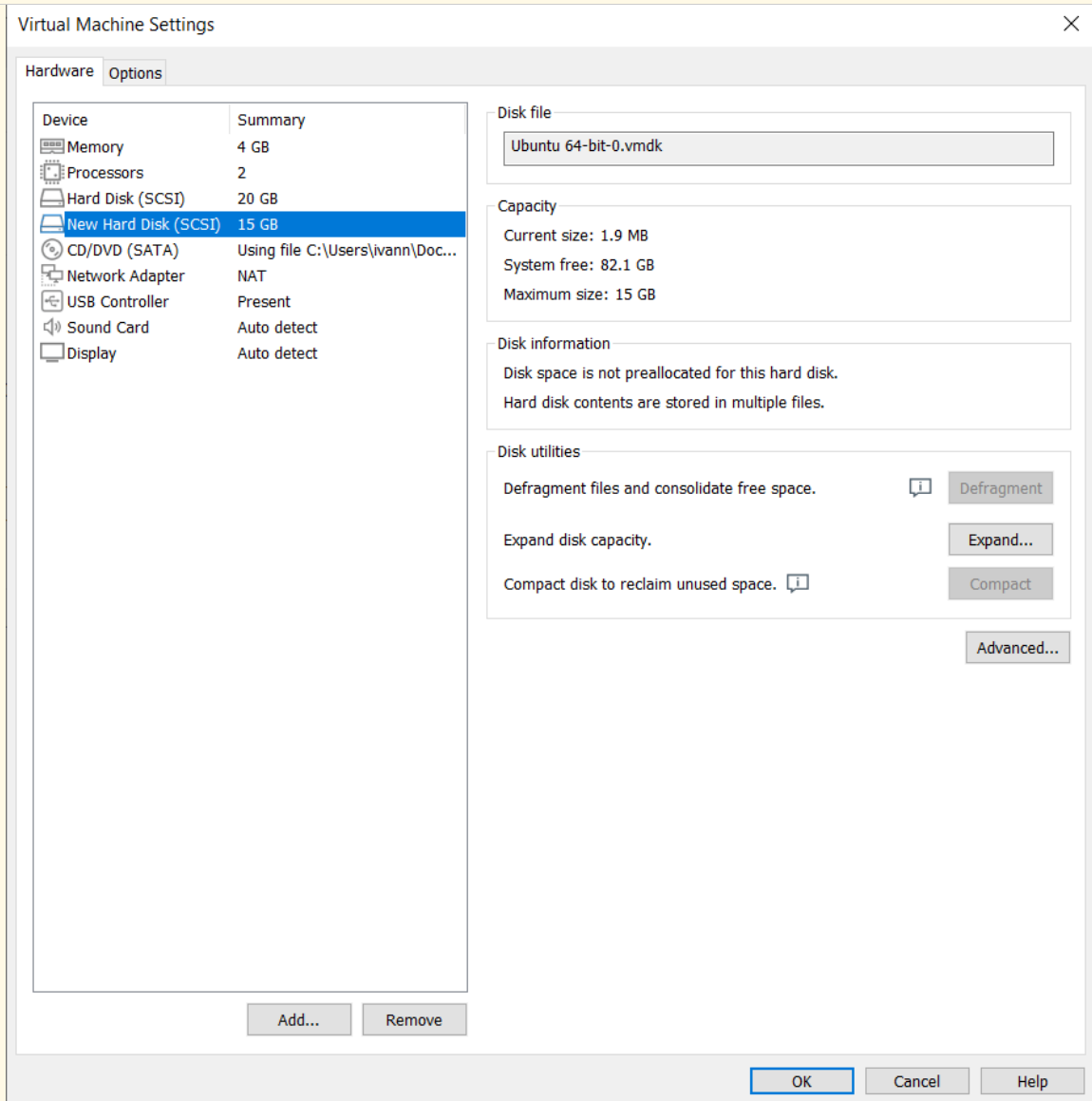
💡 **Astuce :** ⚠️ IMPORTANT : Effectuez cette opération sur un disque VIRTUEL ajouté, pas sur le disque système !

Étapes détaillées

Étape 1 : Ajouter un disque dans VMware

- VM Settings → Add → Hard Disk → Create new virtual disk
- Taille : 15 Go
- Type : SCSI recommandé

Configuration VMware : Ajout du disque de 15 Go



Étape 2 : Identifier le nouveau disque

 **Commande :** `sudo fdisk -l`

 **Description :** Liste tous les disques. Identifiez le nouveau disque (probablement /dev/sdb).

Résultat de fdisk -l montrant le nouveau disque

```
ivan@Srvubu22-virtual-machine: ~  
  
Disque /dev/loop8 : 485,52 MiB, 509100032 octets, 994336 secteurs  
Unités : secteur de 1 × 512 = 512 octets  
Taille de secteur (logique / physique) : 512 octets / 512 octets  
taille d'E/S (minimale / optimale) : 512 octets / 512 octets  
  
Disque /dev/loop13 : 12,32 MiB, 12922880 octets, 25240 secteurs  
Unités : secteur de 1 × 512 = 512 octets  
Taille de secteur (logique / physique) : 512 octets / 512 octets  
taille d'E/S (minimale / optimale) : 512 octets / 512 octets  
  
Disque /dev/loop11 : 53,26 MiB, 55844864 octets, 109072 secteurs  
Unités : secteur de 1 × 512 = 512 octets  
Taille de secteur (logique / physique) : 512 octets / 512 octets  
taille d'E/S (minimale / optimale) : 512 octets / 512 octets  
  
Disque /dev/loop14 : 452 KiB, 462848 octets, 904 secteurs  
Unités : secteur de 1 × 512 = 512 octets  
Taille de secteur (logique / physique) : 512 octets / 512 octets  
taille d'E/S (minimale / optimale) : 512 octets / 512 octets  
ivan@Srvubu22-virtual-machine:~$
```

Étape 3 : Partitionner le disque

 **Commande :** `sudo cfdisk /dev/sdb`

 **Description :** Ouvre l'éditeur de partitions interactif.

Dans cfdisk :

- Sélectionner 'gpt' comme type de table de partition
- Créer partition 1 : 5 Go, type 'Microsoft basic data' (pour NTFS)
- Créer partition 2 : 5 Go, type 'Linux filesystem'
- Créer partition 3 : 5 Go, type 'Linux filesystem'
- Write → yes (pour enregistrer)
- Quit

Interface cfdisk avec les 3 partitions créées

```
ivan@Srvubu22-virtual-machine: ~  
Disque : /dev/sdb  
Size: 15 GiB, 16106127360 bytes, 31457280 sectors  
Étiquette : gpt, identifiant : 57216C00-2A68-B342-A34D-7E79E9DF598D  


| Périphérique | Début    | Fin      | Secteurs | Taille | Type                      |
|--------------|----------|----------|----------|--------|---------------------------|
| /dev/sdb1    | 2048     | 10487807 | 10485760 | 5G     | Données de base Microsoft |
| /dev/sdb2    | 10487808 | 20973567 | 10485760 | 5G     | Système de fichiers Linux |
| >> /dev/sdb3 | 20973568 | 31457246 | 10483679 | 5G     | Système de fichiers Linux |

  
UUID de la partition : BC01A817-EC2D-8141-84BE-B6C895D108CD  
Type de la partition : Système de fichiers Linux (0FC63DAF-8483-4772-8E7  
[ Supprimer ] [ Resize ] [ Quitter ] [ Type ] [ Aide ]  
[ Écrire ] [Sauvegarder]  
Écrire la table de partitions sur le disque (des données peuvent être détruites)
```

Étape 4 : Formater les partitions

 **Commande :** `sudo mkfs.ntfs /dev/sdb1`

 **Description :** Formate la partition 1 en NTFS (pour Windows).

 **Commande :** `sudo mkfs.ext4 /dev/sdb2`

 **Description :** Formate la partition 2 en ext4 (Linux).

 **Commande :** `sudo mkfs.ext4 /dev/sdb3`

 **Description :** Formate la partition 3 en ext4 (Linux).

 **Astuce :** Si `mkfs.ntfs` n'est pas disponible, installez-le avec : `sudo apt install ntfs-3g`

 **Formatage des 3 partitions**


```

ivan@Srvubu22-virtual-machine: ~
En train de créer un système de fichiers avec 1310720 4k blocs et 327680 i-noeuds.
UUID de système de fichiers=b38552e1-7d19-4dcc-93fb-25734918810b
Superblocs de secours stockés sur les blocs :
    32768, 98304, 163840, 229376, 294912, 819200, 884736

Allocation des tables de groupe : complété
Écriture des tables d'i-noeuds : complété
Création du journal (16384 blocs) : complété
Écriture des superblocs et de l'information de comptabilité du système de
fichiers : complété

ivan@Srvubu22-virtual-machine:~$ sudo mkfs.ext4 /dev/sdb3
mke2fs 1.46.5 (30-Dec-2021)
En train de créer un système de fichiers avec 1310459 4k blocs et 327680 i-noeuds.
UUID de système de fichiers=c2c2fdd2-bcfc-4d56-8924-19cfb62e1110
Superblocs de secours stockés sur les blocs :
    32768, 98304, 163840, 229376, 294912, 819200, 884736

Allocation des tables de groupe : complété
Écriture des tables d'i-noeuds : complété
Création du journal (16384 blocs) : complété
Écriture des superblocs et de l'information de comptabilité du système de
fichiers : complété

ivan@Srvubu22-virtual-machine:~$

```

Étape 5 : Créer les points de montage

 **Commande :** `sudo mkdir -p /mnt/windows /mnt/linux1 /mnt/linux2`

 **Description :** Crée les répertoires qui serviront de points de montage.

Création des points de montage

```

ivan@Srvubu22-virtual-machine:~$ sudo mkdir -p /mnt/windows /mnt/linux1 /mnt/linux2
ivan@Srvubu22-virtual-machine:~$

```

Étape 6 : Monter les partitions

 **Commande :** `sudo mount /dev/sdb1 /mnt/windows`

`sudo mount /dev/sdb2 /mnt/linux1`

`sudo mount /dev/sdb3 /mnt/linux2`

 **Description :** Monte chaque partition sur son point de montage respectif.

 **Commande :** `df -h | grep sdb`

 **Description :** Vérifie que les partitions sont bien montées.

```
ivan@Srvubu22-virtual-machine:~$ sudo mkdir -p /mnt/windows /mnt/linux1 /mnt/linux2
ivan@Srvubu22-virtual-machine:~$ sudo mount /dev/sdb1 /mnt/windows
sudo mount /dev/sdb2 /mnt/linux1
sudo mount /dev/sdb3 /mnt/linux2
ivan@Srvubu22-virtual-machine:~$ df -h
Sys. de fichiers Taille Utilisé Dispo Uti% Monté sur
tmpfs            387M   2,0M  385M   1% /run
/dev/sda3         20G    13G   5,8G  69% /
tmpfs            1,9G     0   1,9G   0% /dev/shm
tmpfs            5,0M    4,0K   5,0M   1% /run/lock
/dev/sda2         512M    6,1M  506M   2% /boot/efi
tmpfs            387M   104K  387M   1% /run/user/1000
/dev/sr0          4,7G    4,7G     0 100% /media/ivan/Ubuntu 22.04.3 LTS amd64
/dev/sdb1         5,0G    27M   5,0G   1% /mnt/windows
/dev/sdb2         4,9G    24K   4,6G   1% /mnt/linux1
/dev/sdb3         4,9G    24K   4,6G   1% /mnt/linux2
ivan@Srvubu22-virtual-machine:~$
```

Étape 7 : Tester l'écriture

Commandes de test :

```
# Test partition Windows (NTFS)
sudo touch /mnt/windows/test_windows.txt
echo "Test NTFS" | sudo tee /mnt/windows/test_windows.txt
cat /mnt/windows/test_windows.txt

# Test partition Linux 1
sudo touch /mnt/linux1/test_linux1.txt
echo "Test ext4 partition 1" | sudo tee /mnt/linux1/test_linux1.txt
cat /mnt/linux1/test_linux1.txt

# Test partition Linux 2
sudo touch /mnt/linux2/test_linux2.txt
echo "Test ext4 partition 2" | sudo tee /mnt/linux2/test_linux2.txt
cat /mnt/linux2/test_linux2.txt

# Vérifier tous les fichiers créés
ls -l /mnt/windows/ /mnt/linux1/ /mnt/linux2/
```

 **Création et vérification des fichiers de test sur chaque partition**

```

echo "Test ext4 partition 2" | sudo tee /mnt/linux2/test_linux2.txt
cat /mnt/linux2/test_linux2.txt

# Vérifier tous les fichiers créés
ls -l /mnt/windows/ /mnt/linux1/ /mnt/linux2/
Test NTFS
Test NTFS
Test ext4 partition 1
Test ext4 partition 1
Test ext4 partition 2
Test ext4 partition 2
/mnt/linux1/:
total 20
drwx----- 2 root root 16384 févr.  8 16:35 lost+found
-rw-r--r--  1 root root   22 févr.  8 16:38 test_linux1.txt

/mnt/linux2/:
total 20
drwx----- 2 root root 16384 févr.  8 16:35 lost+found
-rw-r--r--  1 root root   22 févr.  8 16:38 test_linux2.txt

/mnt/windows/:
total 1
-rwxrwxrwx  1 root root 10 févr.  8 16:38 test_windows.txt
ivan@Srvubu22-virtual-machine:~$

```

Étape 8 : Montage automatique au démarrage (optionnel)

Pour que les partitions soient montées automatiquement au démarrage, modifiez /etc/fstab :

 **Commande :** `sudo nano /etc/fstab`

 **Description :** Ouvre le fichier de configuration des montages.

Ajoutez ces lignes :

```

/dev/sdb1 /mnt/windows ntfs-3g defaults 0 2
/dev/sdb2 /mnt/linux1 ext4      defaults 0 2
/dev/sdb3 /mnt/linux2 ext4      defaults 0 2

```

 **Astuce :**  Attention : Une erreur dans /etc/fstab peut empêcher le démarrage du système. Vérifiez bien avant de redémarrer.

 **Fichier /etc/fstab avec les entrées de montage**

```

ivan@Srvubu22-virtual-machine:~$ sudo su
[sudo] Mot de passe de ivan :
root@Srvubu22-virtual-machine:/home/ivan# /etc/fstab
bash: /etc/fstab: Permission non accordée
root@Srvubu22-virtual-machine:/home/ivan# cat /etc/fstab
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point> <type> <options>        <dump>  <pass>
# / was on /dev/sda3 during installation
UUID=b88a6000-1485-49e2-9209-12e1c27ecd50 /          ext4      errors=remount-ro 0 1
# /boot/efi was on /dev/sda2 during installation
UUID=EB52-C7BC /boot/efi   vfat      umask=0077      0 1
/swapfile                                 none      swap       sw 0 0
/dev/sdb1 /mnt/windows ntfs-3g defaults 0 2
/dev/sdb2 /mnt/linux1  ext4      defaults 0 2
/dev/sdb3 /mnt/linux2  ext4      defaults 0 2
root@Srvubu22-virtual-machine:/home/ivan#

```